অধ্যায় ২৫

JavaScript Backend Development with Node.js

REST APIs, Express, Databases, Authentication এবং Production Architecture

এই অধ্যায়ের লক্ষ্য

এই অধ্যায় শেষে পাঠক Node.js runtime, HTTP request–response, RESTful API, Express middleware, input validation, password hashing, session authentication, authorization, database persistence, deployment security ও backend testing-এর মৌলিক ধারণা ব্যাখ্যা করতে এবং একটি ছোট API তৈরি করতে পারবেন। আগের অধ্যায়ের TypeScript domain model-এর সঙ্গে এই অধ্যায়ের transport ও persistence ধারণা যুক্ত করা যাবে।

২৫.১ Browser JavaScript বনাম Node.js

Browser-এ JavaScript DOM, `window` ও browser storage পায়। Node.js-এ filesystem, network server, process এবং server-side APIs পাওয়া যায়। Backend-এর দায়িত্ব শুধু JSON পাঠানো নয়; identity, business rules, authorization, data integrity, error handling ও operational safety বজায় রাখা।

```javascript
// hello.mjs
console.log('Node version:', process.version);
console.log('Environment:', process.env.NODE_ENV ?? 'development');
```

Node.js-এ CommonJS (`require`, `module.exports`) এবং ES Modules (`import`, `export`) উভয় ব্যবস্থা আছে। এই বইয়ের project-এ ES Modules ব্যবহার করা হয়েছে। `package.json`-এ `"type": "module"` দিলে `.js` ফাইলও ESM হতে পারে; `.mjs` নিজে থেকেই ESM।

২৫.২ HTTP এবং REST

HTTP request-এর Method, URL, Headers ও Body থাকে। Response-এর Status Code, Headers ও Body থাকে। API resource-oriented হলে endpoint বোঝা সহজ হয়।

Method	Endpoint	কাজ	সাধারণ Status
GET	`/api/packages`	Package list	200
POST	`/api/register`	Account তৈরি	201
POST	`/api/login`	Session তৈরি	200 বা 401
GET	`/api/bookings`	নিজের Booking list	200 বা 401
POST	`/api/bookings`	Booking তৈরি	201 বা 400

Method	Endpoint	কাজ	সাধারণ Status
PATCH	`/api/bookings/:id/cancel`	Booking বাতিল	200, 404 বা 409

`400` ভুল input, `401` unauthenticated, `403` authenticated কিন্তু permission নেই, `404` resource নেই, `409` conflict, `500` unexpected server failure বোঝায়। `401` ও `403` এক নয়। API design-এ একই ধরনের error-এর জন্য একই response shape ব্যবহার করা সুবিধাজনক।

২৫.৩ HTTP Server-এর মূল ধারণা

Node.js built-in `node:http` দিয়েও API বানানো যায়। Request body streaming হওয়ায় আকার সীমিত করতে হবে; `JSON.parse()` ব্যর্থ হলে `400` দিতে হবে।

JAVASCRIPT

```
import http from 'node:http';
const server = http.createServer((req, res) => {
  if (req.method === 'GET' && req.url === '/health') {
    res.writeHead(200, { 'content-type': 'application/json' });
    res.end(JSON.stringify({ status: 'ok' }));
    return;
  }
  res.writeHead(404, { 'content-type': 'application/json' });
  res.end(JSON.stringify({ error: 'NOT_FOUND' }));
});
server.listen(3000);
```

এটি HTTP lifecycle শেখার উদাহরণ। Full Project-এ আলাদা Router-style Handler, Service এবং Repository রাখা হয়েছে।

২৫.৪ Express কী সুবিধা দেয়?

Express routing ও middleware chain সহজ করে। নিচের snippet চালাতে `npm install express` প্রয়োজন। এটি সম্পূর্ণ TravelCore project-এর runtime dependency নয়; project-টি পরীক্ষাযোগ্য রাখতে built-in HTTP ব্যবহার করা হয়েছে।

JAVASCRIPT

```
import express from 'express';
const app = express();
app.use(express.json({ limit: '16kb' }));
app.get('/health', (_req, res) => res.json({ status: 'ok' }));
app.use((err, _req, res, _next) => {
  console.error(err);
  res.status(500).json({ error: 'INTERNAL_ERROR' });
});
app.listen(3000);
```

Middleware সাধারণত logging, authentication, JSON parsing, request validation ও error handling-এর জন্য ব্যবহৃত হয়। Route-এর আগে middleware বসানোর ক্রম গুরুত্বপূর্ণ। Express error middleware-এ চারটি parameter থাকে।

২৫.৫ Layered Architecture

```
HTTP Client
    ↓
Router / Controller  → HTTP parsing, status, headers
    ↓
Service              → business rules, authorization
    ↓
Repository           → data access contract
    ↓
Database             → durable state and constraints
```

Controller-এর মধ্যে SQL, password hashing, discount policy সব একসঙ্গে রাখলে test ও maintenance কঠিন হয়। Service-এ domain rule, Repository-তে storage operation রাখা ভালো। Dependency injection করলে in-memory repository দিয়ে unit test চালানো যায়।

২৫.৬ Input Validation এবং Price Integrity

Frontend-এর input বিশ্বাস করা যাবে না। Client থেকে price নেওয়ার বদলে server catalog থেকে price নির্ধারণ করবে। Quantity integer কি না, ID সঠিক কি না, payload size সীমার মধ্যে কি না যাচাই করতে হবে।

```javascript
export function validateTravelers(value) {
  if (!Number.isSafeInteger(value) || value < 1 || value > 20) {
    throw new Error('Travelers must be 1–20');
  }
  return value;
}
```

Validation-এর দুই স্তর: syntax/shape validation এবং business-rule validation। যেমন `travelers=2` shape-এ বৈধ, কিন্তু কোনো নির্দিষ্ট package-এ ২ জনের booking বন্ধ থাকলে business rule-এ বাতিল হতে পারে।

২৫.৭ Authentication বনাম Authorization

Authentication নির্ধারণ করে **কে** request করছেন। Authorization নির্ধারণ করে **তিনি কী করতে পারবেন**। Login থাকা মানেই অন্য ব্যবহারকারীর booking দেখা বা বাতিল করার অনুমতি নয়। Project-এ `userId` দিয়ে নিজের booking filter করা হয়।

Password plaintext রাখা যাবে না। এই educational project-এ Node.js `scryptSync` + random salt দিয়ে password hash করা হয়েছে। Production-এ Argon2id-এর মতো সুপারিশকৃত password hashing ব্যবস্থা এবং উপযুক্ত cost parameters বিবেচনা করা উচিত। Password যাচাইয়ে timing-safe comparison ব্যবহার করা হয়েছে।

Session cookie-তে `HttpOnly`, `SameSite`, `Secure` (HTTPS-এ), expiry এবং server-side session invalidation প্রয়োজন। এই demo-তে token server memory-তে থাকে; restart হলে session হারিয়ে যায়। Cookie-based state-changing endpoints-এর জন্য production-এ CSRF protection দরকার। Login rate limiting, account lockout policy এবং audit logging-ও প্রয়োজন।

২৫.৮ Database: In-memory থেকে PostgreSQL

এই অধ্যায়ের runnable example-এ `Map`-ভিত্তিক Repository ব্যবহার করা হয়েছে যাতে external database ছাড়া project পরীক্ষা করা যায়। এটি durable database নয়। Real production-এ relational schema ও constraints দরকার।

```sql
CREATE TABLE users (
  id UUID PRIMARY KEY,
  email TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL
);

CREATE TABLE bookings (
  id UUID PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES users(id),
  package_id TEXT NOT NULL,
  travelers INTEGER NOT NULL CHECK (travelers BETWEEN 1 AND 20),
  total_bdt INTEGER NOT NULL CHECK (total_bdt >= 0),
  status TEXT NOT NULL CHECK (status IN ('pending','confirmed','cancelled')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX bookings_user_created_idx ON bookings(user_id, created_at DESC);
```

Schema migration version-controlled হতে হবে। SQL query parameterized হতে হবে, string concatenation দিয়ে untrusted data SQL-এ ঢোকানো যাবে না। Booking + inventory update একই transaction-এ করতে হতে পারে; rollback, isolation ও idempotency design জরুরি। পরের অধ্যায়ে Prisma, transactions ও query optimization বিস্তারিত থাকবে।

২৫.৯ API Security ও Production Checklist

- HTTPS, reverse proxy, secure cookie, CSRF protection, rate limiting ও request-size limit।
- CORS কেবল প্রয়োজনীয় trusted origins-এ সীমিত; CORS নিজে authentication নয়।
- Secrets source code বা Git repository-তে নয়; environment/secret manager-এ।
- Logs-এ password, session token, sensitive user data নয়।
- `npm audit`, dependency updates, least-privilege database role ও backups।
- Graceful shutdown, health checks, metrics, tracing এবং incident response plan।
- Horizontal scaling-এর ক্ষেত্রে in-memory session বা process-local data যথেষ্ট নয়।

২৫.১০ Practical Project — TravelCore Booking Backend

লক্ষ্য ও Folder Structure

```
source-code/chapter-25/
├── package.json
├── catalog.mjs
├── core.mjs
├── repository.mjs
├── server.mjs
├── core.test.mjs
├── http.test.mjs
└── README.md
```

বইয়ের Companion Source Code-এ উপরোক্ত ফাইলগুলোর **সম্পূর্ণ runnable source code** আছে। এখানে গুরুত্বপূর্ণ অংশ ব্যাখ্যা করছি; source code বাদ দেওয়া হয়নি—আলাদা project directory-তে সংরক্ষিত।

Step 1: `cd source-code/chapter-25` তারপর `npm test`। **Step 2:** `npm start`। **Step 3:** `http://localhost:3000/health` এবং `/api/packages` খুলে দেখো। **Step 4:** `POST /api/register` দিয়ে account তৈরি, `POST /api/login` দিয়ে cookie নাও। **Step 5:** সেই cookie দিয়ে `POST /api/bookings` এবং `GET /api/bookings` চালাও।

Request Example

TERMINAL

```
curl -i -c cookies.txt -H 'Content-Type: application/json' \
  -d '{"email":"reader@example.com","password":"a strong password 123"}' \
  http://localhost:3000/api/register

curl -i -c cookies.txt -H 'Content-Type: application/json' \
  -d '{"email":"reader@example.com","password":"a strong password 123"}' \
  http://localhost:3000/api/login

curl -i -b cookies.txt -H 'Content-Type: application/json' \
  -d '{"packageId":"cox","travelers":2}' \
  http://localhost:3000/api/bookings
```

Windows PowerShell-এ `curl` alias থাকলে `curl.exe` ব্যবহার করো। Demo-এর cookie file সংবেদনশীল; অন্যের সঙ্গে share করবে না।

Project Design Decisions

- **Catalog-based price:** client arbitrary total দিতে পারে না।
- **Password hashing:** salted scrypt; plaintext password repository-তে নেই।
- **Authorization:** booking owner ছাড়া অন্য user cancel করতে পারে না।
- **Error format:** `{ error, message }`।
- **Session:** cryptographically random token; server-side hashed token lookup।
- **Testability:** Service pure-ish dependency-injected repository-তে কাজ করে।

সীমাবদ্ধতা

এই Demo-তে real database, atomic concurrent uniqueness, persistent session, CSRF defense, rate limiting ও production-grade observability নেই। তাই এটিকে Production Booking Backend হিসেবে deploy করবে না।

২৫.১১ Common Mistakes এবং Debugging

Mistake 1: Frontend price বিশ্বাস করা। **Fix:** server catalog থেকে হিসাব।

Mistake 2: `401` ও `403` এক করা। **Fix:** login এবং permission আলাদা পরীক্ষা।

Mistake 3: password log করা। **Fix:** secrets redact করা।

Mistake 4: `await` বাদ দেওয়া। **Fix:** asynchronous repository ও errors পরীক্ষা।

Mistake 5: শুধু unit test করে database concurrency safe দাবি করা। **Fix:** real database integration ও transaction test।

Mistake 6: cookie session ব্যবহার করে CSRF threat ভুলে যাওয়া। **Fix:** origin checks, CSRF token, SameSite এবং threat model।

২৫.১২ অনুশীলনী (১০টি)

1. `GET /health`-এ uptime যোগ করো।
2. `POST /api/bookings`-এ `travelers: 0` দিলে কী status হওয়া উচিত?
3. একই email দিয়ে দুইবার registration করলে conflict কেন?
4. `PATCH /api/bookings/:id/cancel`-এ owner check কেন দরকার?
5. `GET /api/bookings`-এ pagination design করো।
6. Request body size limit না থাকলে কী ঝুঁকি?
7. Password hash-এর সঙ্গে salt কেন লাগে?
8. `401` এবং `403`-এর ব্যবহার উদাহরণ দাও।
9. In-memory Repository-কে PostgreSQL-এ বদলানোর interface লিখো।
10. CSRF protection ও rate limiting-এর integration test plan তৈরি করো।

২৫.১৩ Sample Solutions

1. `process.uptime()` seconds response-এ দাও; এটি process uptime, service availability নয়।
2. `400 INVALID_TRAVELERS`; quantity validation service-এ থাকবে।
3. Duplicate identity prevent করতে `409 EMAIL_EXISTS`; real DB-তে `UNIQUE(email)` constraint অপরিহার্য।
4. Booking ID জানা থাকলেও অন্য user-এর booking পরিবর্তনের অধিকার নেই; IDOR vulnerability ঠেকায়।
5. `?limit=20&cursor=...`; stable `(created_at,id)` order ও indexed cursor ব্যবহার করো।
6. Unbounded memory/CPU usage ও denial-of-service ঝুঁকি।
7. একই password-এর hash ভিন্ন রাখতে ও precomputed rainbow table আক্রমণ কঠিন করতে।

8. Session না থাকলে `401`; login থাকা সত্ত্বেও admin-only action করলে `403`।
9. `findUserByEmail`, `saveUser`, `saveBooking`, `findBooking`, `listBookings`—এই contract-এর async PostgreSQL implementation; transaction ও constraints যোগ করো।
10. Cross-site POST reject, valid CSRF token accept, repeated login attempts throttle, legitimate user unaffected—এই case-গুলো test করো।

২৫.১৪ MCQ — Final Check

1. Node.js-এ HTTP server তৈরির built-in module কোনটি? (A) `node:http` (B) `window` (C) `document` (D) `localStorage`
2. সফল resource creation-এর সাধারণ status? (A) 404 (B) 201 (C) 500 (D) 302
3. `401` সাধারণত কী বোঝায়? (A) No authentication (B) No storage (C) Success (D) Redirect
4. User A-এর booking User B cancel করতে না পারা কোন নীতি? (A) CSS (B) Authorization (C) Minification (D) Hoisting
5. Password plaintext না রেখে কী রাখা উচিত? (A) Base64 (B) Hash + Salt (C) URL (D) Cookie Name
6. SQL Injection কমাতে কী দরকার? (A) String concatenation (B) Parameterized query (C) `eval` (D) `innerHTML`
7. HTTP request body size limit কেন? (A) Font (B) Resource abuse prevent (C) Syntax color (D) SEO
8. CSRF মূলত কোন context-এ গুরুত্বপূর্ণ? (A) Cookie-authenticated state changes (B) Static comments (C) Array sort (D) CSS animation
9. Database transaction কেন? (A) Atomic consistency (B) Font loading (C) DOM rendering (D) Code highlighting
10. `Map`-based repository restart হলে কী হয়? (A) Data persists (B) Data disappears (C) Migrates to SQL (D) Auto backup

Answer Key ও Explanation

1. **A** — `node:http` Node.js-এর HTTP module।
2. **B** — নতুন resource তৈরিতে `201 Created`।
3. **A** — request-এ বৈধ authentication নেই।
4. **B** — resource ownership authorization rule।
5. **B** — salted, computationally expensive password hashing।
6. **B** — parameter binding data ও SQL command আলাদা রাখে।
7. **B** — অস্বাভাবিক বড় payload-এর resource exhaustion ঠেকায়।
8. **A** — browser স্বয়ংক্রিয়ভাবে cookie পাঠাতে পারে।
9. **A** — একাধিক dependent database operation একসঙ্গে commit/rollback।
10. **B** — process memory durable storage নয়।

২৫.১৫ Chapter Summary ও পরবর্তী অধ্যায়

Node.js দিয়ে backend HTTP request handle করে; Express middleware ও routing সহজ করে; Service business logic রাখে; Repository persistence abstract করে; Authentication ও Authorization আলাদা; Database constraints এবং security review ছাড়া demo production-ready নয়। পরের অধ্যায় **Advanced Database Integration, ORM, Query Optimization এবং Scalable Data Architecture**—এ PostgreSQL ও Prisma নিয়ে কাজ করা হবে।